TRAKYA ÜNİVERSİTESİ · BİLGİSAYAR MÜHENDİSLİĞİ

Dosya Organizasyonu

Hafta 1 · Giriş: Dosya Yapılarının Tasarımı

Doç. Dr. Emir Öztürk

2025/2026 Güz Yarıyılı

DOSYA ORGANIZASYONU

İletişim



Doç. Dr. Emir Öztürk

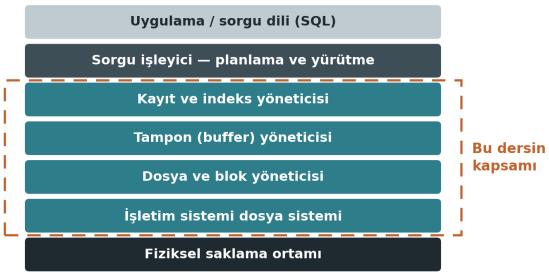
Trakya Üniversitesi
Mühendislik Fakültesi · Bilgisayar Mühendisliği

İLETIŞIM

emirozturk@trakya.edu.tr **WhatsApp: @emirozturk**

KAPSAM

Dersin sistem içindeki yeri



Bu dersin alanı: kayıt, indeks, tampon, blok

Ölçüt: diskteki yerleşim ve erişim maliyeti

İşletim sistemi dosya sistemleri de kapsamda

Veri yönetim sistemleri katmanlı bir mimariye sahiptir. En üstte kullanıcının veriyi mantıksal düzeyde tanımladığı sorgu dili, en altta ise verinin fiziksel olarak saklandığı donanım bulunmaktadır. Sorgu işleyici, bildirimsel bir sorguyu yürütülebilir bir plana dönüştürmektedir. Kayıt ve indeks yöneticisi bu planın gerektirdiği erişim yollarını sağlamaktadır. Tampon yöneticisi ana bellek ile disk arasındaki blok aktarımını yönetmektedir. Dosya ve blok yöneticisi ise saklama ortamıyla doğrudan etkileşimde bulunmaktadır. Bu derste, sorgu işlemenin üzerindeki katmanlar kapsam dışı bırakılmakta ve verinin diskteki düzeni ile bu düzenin erişim maliyeti üzerindeki etkisi incelenmektedir.

14 HAFTALIK PLAN · ILK YEDI HAFTA

Ders içeriği

1	**Giriş**	Tasarım ve tarihsel gelişim
2	**Depolama donanımı**	Disk, flash bellek, hiyerarşi
3	**Dosya işlemleri**	Temel işlemler, sayfa, tampon
4	**Kayıt biçimleri**	Sabit ve değişken uzunluk
5	**Dosya sistemi temelleri**	Inode, dizin, blok tahsisi
6	**Gerçek dosya sistemleri**	FFS, günlükleme, ext4, NTFS
7	**Erişim organizasyonu**	Sıralı ve doğrudan erişim

14 HAFTALIK PLAN · SON YEDI HAFTA

Ders içeriği

8	**Cosequential processing**	Çok yollu birleştirme, harici sıralama
9	**İndekslemeye giriş**	İndeks kavramı, çoklu anahtar
10	**B-tree**	Çok seviyeli indeks, bölme
11	**B+ tree**	Sıra kümesi, ön ek indeksleri
12	**Hashing**	Statik, extendible, linear
13	**Depolama modelleri**	Sütun bazlı depolama, sıkıştırma
14	**Log-structured yapılar**	LSM ağaçları, modern sistemler

İŞLEYIŞ

Değerlendirme

%30

Ara sınav

%70

Final sınavı

Kod yazımı gerekmiyor

Verilen kodun okunabilmesi gerekli

Algoritmalar ve çözümleri

TARTIŞMAYA AÇIK

Haftalık ders saati düzeni

2 + 2

İki saat teori, iki saat uygulama

Teorik içerik ve kodlama

3 + 1

Üç saat teori, bir saat uygulama

Teorik içerik ve küçük kod parçaları

4 + 0

Dört saat teori

Sadece teorik anlatım

İŞLEYIŞ

Uygulama ve programlama dili

Dil: C++

Bu düzeydeki yazılımların geliştirildiği dil

Beklenen düzey

Kod yazma değil, kod okuma

Kapsam

Sınıf tanımı, dosya akışları, temel veri yapıları

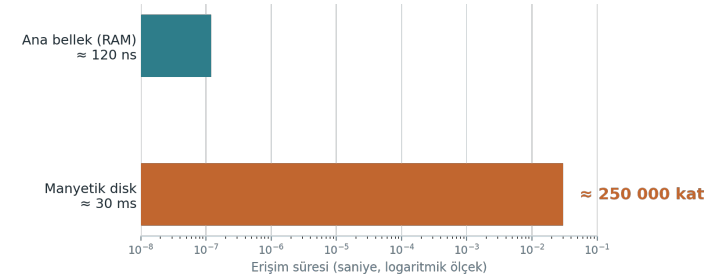
1

Neden dosya yapıları?

Donanımın dayattığı kısıt

TEMEL KISIT

Disk yavaşlığı



Ana bellek ≈ 120 nanosaniye

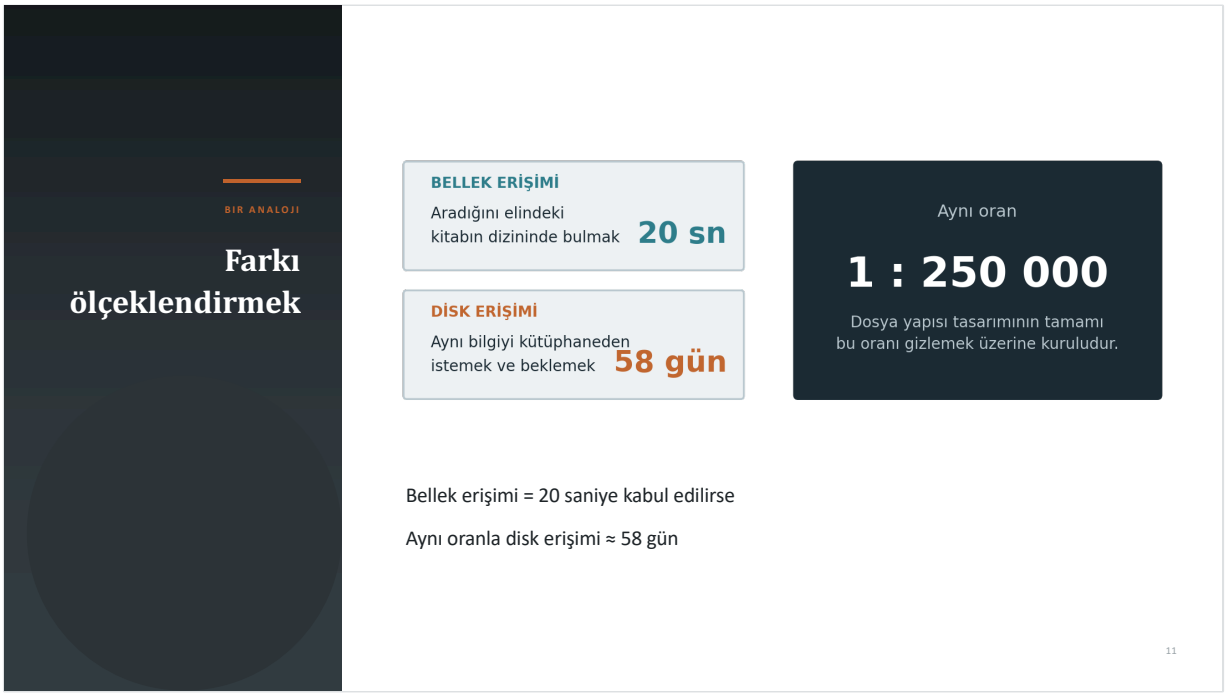
Manyetik disk ≈ 30 milisaniye

Fark ≈ 250 000 kat

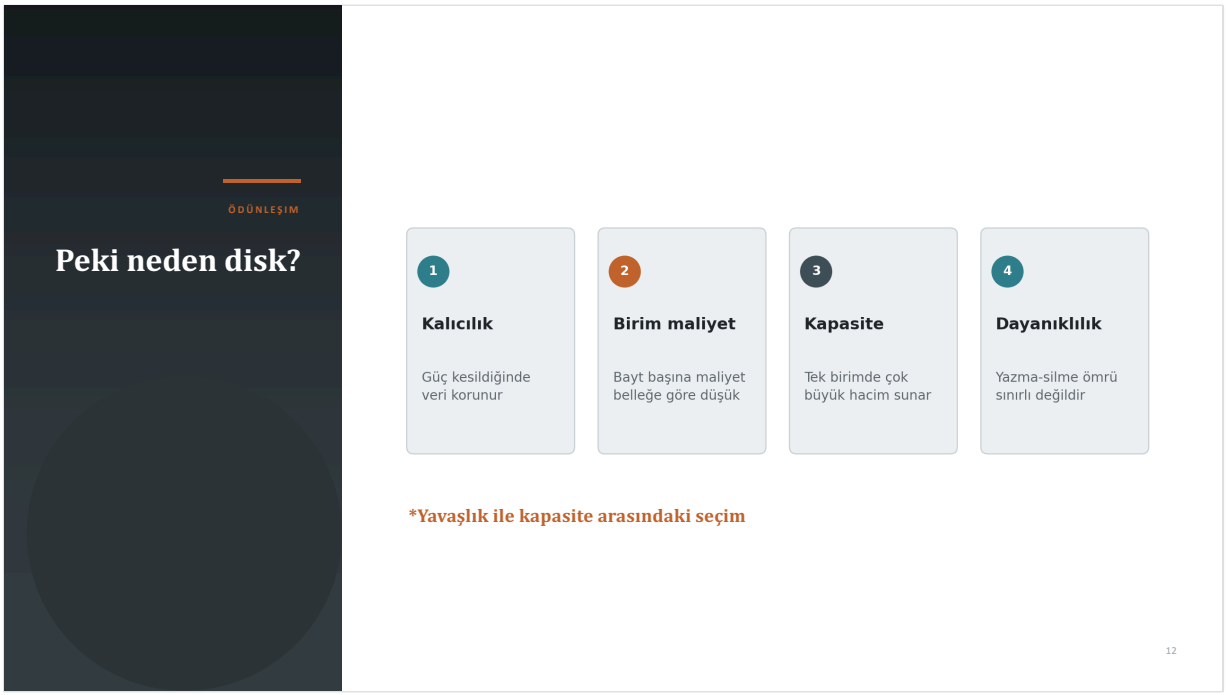
Yazılımla kapatılamaz

– Tek değişken: kaç kez disk erişimi yapıldığı

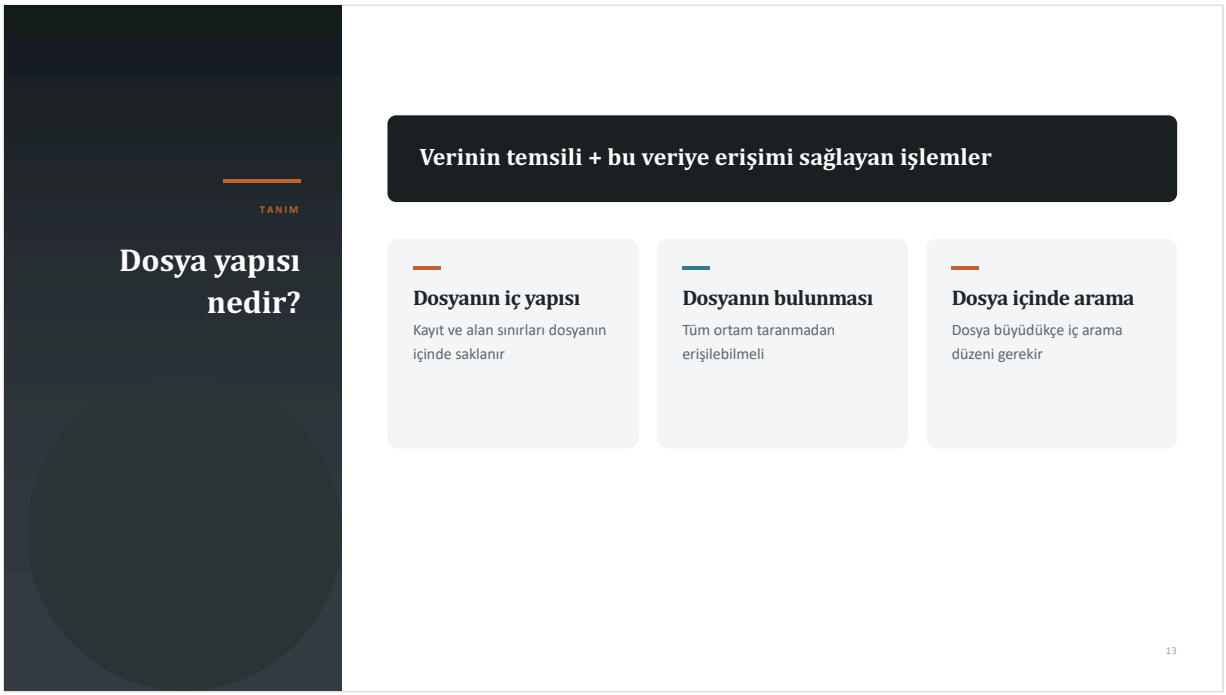
Dosya yapılarına ilişkin bütün tasarım kararlarının çıkış noktası, ana bellek ile kalıcı saklama ortamı arasındaki erişim süresi farkıdır. Ana bellekten bir veriye erişim yaklaşık 120 nanosaniye sürerken, aynı verinin manyetik diskten okunması yaklaşık 30 milisaniye almaktadır. Bu iki değer arasındaki oran yaklaşık 250 000'dir. Söz konusu fark, ortamların fiziksel çalışma ilkelerinden kaynaklandığından yazılım düzeyinde ortadan kaldırılamamaktadır. Buna karşılık, bir işlemin toplam süresi disk erişimi sayısıyla doğru orantılı olduğundan, tasarımcı erişim sayısını azaltarak toplam süreyi belirgin biçimde düşürebilmektedir. Dolayısıyla dosya yapısı tasarımında başarım ölçütü çalıştırılan komut sayısı değil, gerçekleştirilen disk erişimi sayısıdır.



Bellek erişimi, aranan bilginin elde tutulan bir kitabın dizininde bulunmasına benzetilerek 20 saniye kabul edilsin. Aynı oran korunduğunda, disk erişimi aynı bilginin bir kütüphaneden istenmesine ve yaklaşık 58 gün beklenmesine karşılık gelmektedir. Bir işlem için gereken erişim sayısının dörtten ikiye düşürülmesi, aynı ölçekte iki aylık bir bekleme süresinin ortadan kaldırılmasına karşılık gelmektedir.



Erişim süresi bakımından belirgin biçimde dezavantajlı olmasına karşın manyetik diskin kullanımını sürdüren nedenler bulunmaktadır. Disk kalıcı bir ortamdır ve güç kesildiğinde içeriği korunmaktadır. Bayt başına maliyeti ana belleğe ve flash belleğe göre düşük olduğundan, aynı bütçeyle çok daha büyük hacim sunmaktadır. Ayrıca flash belleğin aksine sınırlı bir yazma ve silme ömrü bulunmamaktadır. Bu özelliklerin bir arada bulunması, büyük hacimli ve seyrek erişilen verinin saklanmasında diski hala tercih edilir kılmaktadır.



Dosya yapısı, verinin dosya içindeki temsil biçimi ile bu temsil üzerinde tanımlı okuma, yazma ve değiştirme işlemlerinin bütününü ifade etmektedir. Bu tanımın üç bileşeni bulunmaktadır. Birincisi, dosyanın kendi iç yapısının dosyayla birlikte saklanmasıdır. Kayıtların ve alanların nerede başlayıp nerede bittiği, dosyayı okuyan programın dışarıdan bilmek zorunda olmadığı bir bilgi olmalıdır. İkincisi, istenen dosyanın saklama ortamı üzerinde hızlıca bulunabilmesidir. Bu görev büyük ölçüde dosya sistemine aittir. Üçüncüsü, dosya sayısı ve dosya boyutu büyüdükçe dosyanın içinde de bir arama düzenine gereksinim duyulmasıdır. İndeksleme ve ağaç yapıları bu gereksinimin karşılığıdır.

ÖLÇÜT

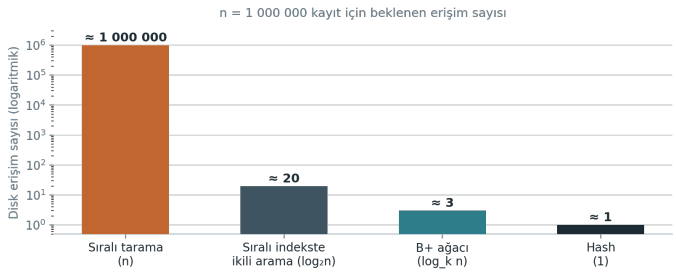
İndekslemenin amacı

İdeal: tek disk erişimi

Gerçekte: olabildiğince az erişim

Ölçüt: erişim sayısının kayıt sayısıyla büyümesi

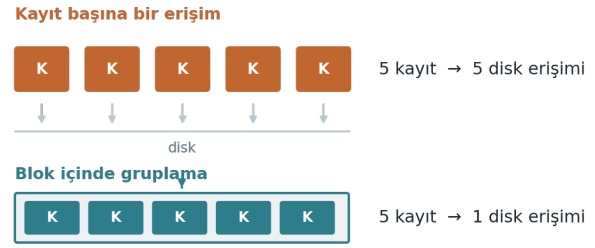
– Sıralı taramadan hash tabanlı erişime doğru yavaşlar



Bir dosya yapısının başarımı, istenen kayda ulaşmak için gereken disk erişimi sayısıyla ölçülmektedir. İdeal durum, aranan bilgiye tek bir erişimle ulaşılmasıdır. Ancak bu hedef genel durumda sağlanamamaktadır. Bu nedenle pratik amaç, erişim sayısının olabildiğince az tutulmasıdır. Değerlendirmede mutlak erişim sayısından çok, bu sayının kayıt sayısıyla birlikte nasıl büyüdüğü önem taşımaktadır. Bir milyon kayıtlık bir dosya için sıralı tarama en kötü durumda bir milyon erişim gerektirirken, sıralı bir indekste ikili arama yaklaşık yirmi, düğüm başına yüz anahtar taşıyan bir B+ ağacı yaklaşık üç, hash tabanlı erişim ise tek erişim gerektirmektedir.

TEMEL FIKIR

Erişim sayısını azaltmak: gruplama



Maliyet: okunan bayt değil, erişim sayısı

Birlikte kullanılan kayıtlar aynı blokta

İkili arama ağacından B-ağacına geçişin gerekçesi

15

Disk erişiminin maliyeti büyük ölçüde konumlandırma süresinden kaynaklandığından, tek bir erişimde okunan bayt sayısının artırılması toplam maliyeti önemli ölçüde değiştirmemektedir. Bu durum, dosya yapısı tasarımının en temel ilkelerinden birini ortaya çıkartır. Birlikte kullanılması olası kayıtlar aynı blok içinde toplanmalıdır. Beş kaydın ortam üzerinde dağınık biçimde durması halinde beş ayrı erişim gerekirken, aynı kayıtların tek bir blokta bulunması durumunda tek bir erişim yeterli olmaktadır. İlgili verinin gruplanması ilkesi, dönem boyunca incelenecek yapıların çoğunda karşımıza çıkacaktır. İkili arama ağacından B tree'ye geçişin gerekçesi de doğrudan bu ilkedir.



Bir dosya yapısının değerlendirilmesinde erişim başarımı tek ölçüt değildir. Kalıcılık, verinin sistem kapatıldıktan sonra da korunmasını gerektirmektedir. Paylaşım, aynı veriye eşzamanlı erişen kullanıcıların birbirlerinin sonuçlarını bozmamasını gerektirmektedir. Bu da eşzamanlılık denetimini zorunlu kılmaktadır. Doğruluk, yarım kalan bir güncellemenin veriyi tutarsız bir durumda bırakmamasını, yani kurtarma düzeneklerinin bulunmasını gerektirmektedir. Ölçeklenebilirlik, yapının kayıt sayısı büyüdüğünde de kullanılabilir kalmasını ifade etmektedir. Kullanılabilirlik ise verinin, erişim ayrıntıları bilinmeden istenebilmesini gerektirmektedir. Bu ölçütler birbiriyle çelişebilmektedir. Tasarım kararları çoğu zaman aralarında bir denge kurmayı gerektirmektedir. Bu çelişki somut olarak şöyle görülmektedir: dayanıklılık için yazılan günlük kayıtları yazma başarımını düşürmekte, eşzamanlı erişim için kullanılan kilitler ise gecikmeyi artırmaktadır.

2

Tarihsel gelişim

Erişim sayısını azaltma çabasının izlediği yol

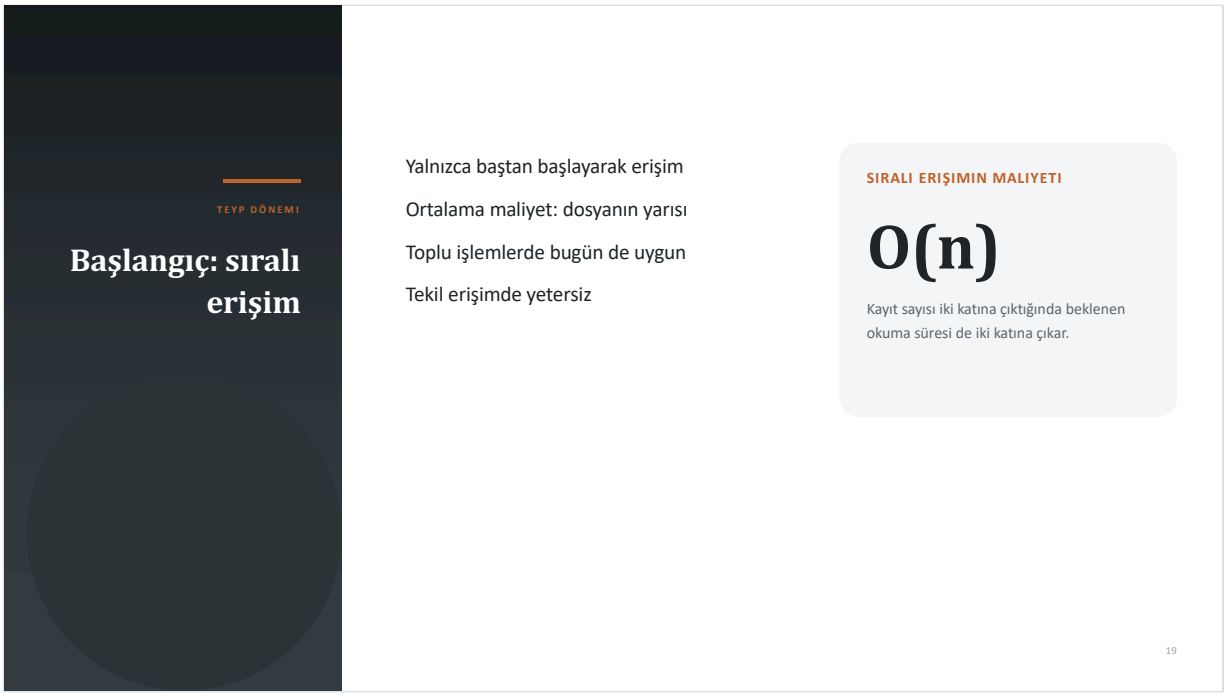
TARIHSEL BAKIŞ

Gelişimin genel görünümü



Her adım, bir öncekinin disk erişim sayısını azaltma çabasından doğmuştur.

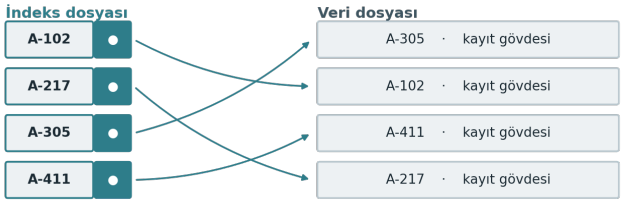
Dosya yapılarının gelişimi, saklama ortamlarının değişimiyle birlikte ilerlemiştir. Teyp döneminde erişim yalnızca sıralı biçimde yapılabildiğinden, dosyalar baştan sona okunmaktaydı. Diskin yaygınlaşmasıyla doğrudan erişim olanaklı hale gelmiş ve indeks listeleri kullanılmaya başlanmıştır. Dosya boyutları büyüdükçe indeksin kendisi de aranması gereken bir yapıya dönüşmüştür. Bu durum, ağaç yapılarının kullanımına yol açmıştır. İkili arama ağaçlarının dengesizleşme sorununa karşılık kendini dengeleyen AVL ağaçları geliştirilmiş, ancak düğüm başına tek anahtar taşınması disk için uygun bulunmamıştır. Bu gereksinim çok yollu ağaçlara, oradan sıralı okumayı da destekleyen B+ ağaçlarına ve tek erişim hedefi doğrultusunda hash tabanlı yapılara götürmüştür.



Dosya yapılarının ilk biçimleri, manyetik teypin çalışma ilkesi tarafından belirlenmiştir. Teypte veriye yalnızca baştan başlayarak sırayla erişilebildiğinden, belirli bir kaydın bulunması ortalama olarak dosyanın yarısının okunmasını gerektirmektedir. Bu maliyet kayıt sayısıyla doğrusal olarak büyümektedir. Söz konusu düzen, dosyaların görece küçük olduğu ve işlemlerin toplu biçimde yürütüldüğü koşullarda yeterli olmuştur. Bordro hesaplama gibi tüm kayıtların zaten okunmasını gerektiren işlemlerde sıralı erişim bugün de uygun bir seçenektir. Ancak dosya boyutlarının büyümesi ve doğrudan erişimli disklerin yaygınlaşmasıyla birlikte, tek tek kayıtlara erişim için bu düzen yetersiz kalmıştır.

DOĞRUDAN ERIŞIM

İndekslerin ortaya çıkışı



İndeks anahtara göre sıralı tutulur; veri dosyası sırasız kalabilir.

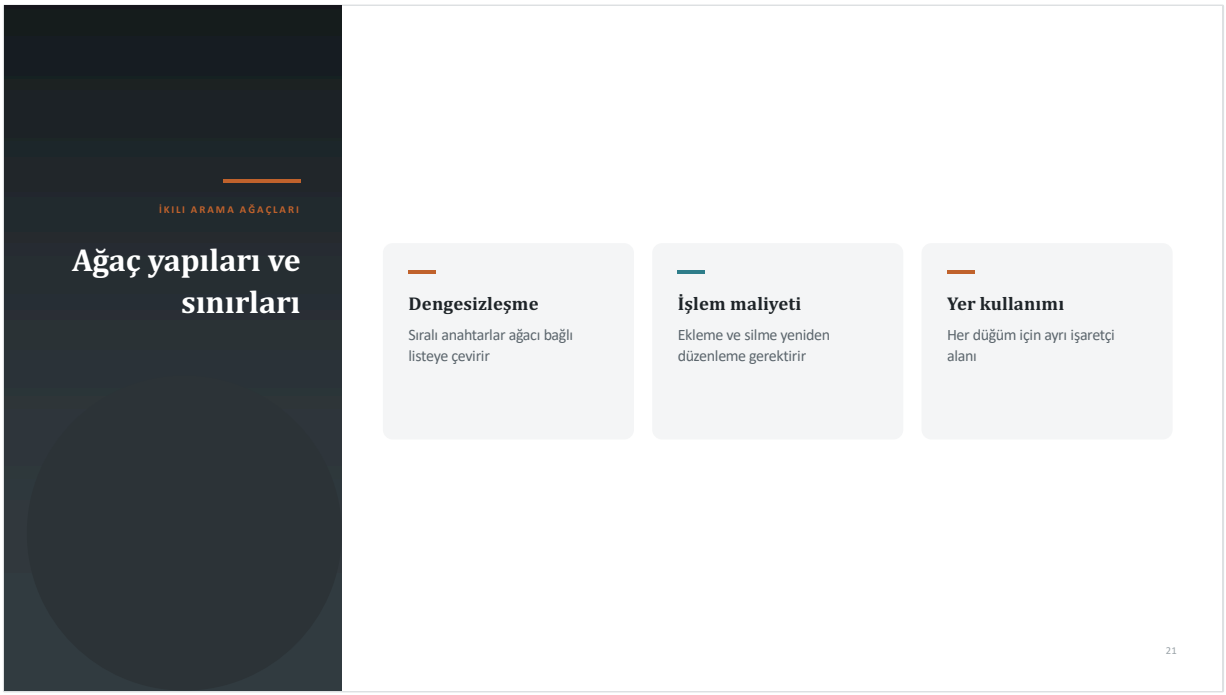
Anahtar değeri → kayıt konumu eşlemesi

Sıralı indekste ikili arama

Veri dosyasının sıralı olması gerekmez

Sınır: indeksin kendisi belleğe sığmaz

İndeks, anahtar değerleri ile bu değerlere karşılık gelen kayıtların dosya içindeki konumlarını eşleyen ikinci bir dosyadır. Anahtara göre sıralı tutulduğundan üzerinde ikili arama yapılabilmektedir. Bu da erişim maliyetini doğrusal büyümeden logaritmik büyümeye indirmektedir. İndeksin sağladığı önemli bir kolaylık, veri dosyasının kendisinin sıralı tutulmasını gereksiz kılmasıdır. Kayıtlar dosyaya eklendikleri sırayla yazılabilmekte, sıralılık yalnızca indeks üzerinde korunmaktadır. Bu ayrım, ekleme ve silme işlemlerinin maliyetini belirgin biçimde azaltmaktadır. Buna karşılık indeks yaklaşımının kendi sınırı bulunmaktadır: dosya büyüdükçe indeksin kendisi de ana belleğe sığmamakta ve bu durumda indeks içinde arama, çözülmesi gereken yeni bir sorun hâline gelmektedir.



İndeksin kendisinde arama sorununa verilen ilk yanıt ağaç yapıları olmuştur. İkili arama ağacında her düğümde bir anahtar tutulmakta ve arama her adımda olası kayıtların yarısını elemektedir. Ancak bu yapının üç güçlüğü bulunmaktadır. Birincisi, anahtarların sıralı geldiği durumlarda ağacın dengesizleşerek bağlı listeye dönüşmesi ve arama maliyetinin yeniden doğrusal hâle gelmesidir. İkincisi, ekleme ve silme işlemlerinin ağacın yeniden düzenlenmesini gerektirebilmesi ve bu düzenlemenin diskte pahalı olmasıdır. Üçüncüsü, her düğüm için ayrı işaretçi alanı ayrılmasının ek yer maliyeti doğurmasıdır. Bu güçlükler, kendini dengeleyen ağaç yapılarına yönelik çalışmaları başlatmıştır.

AVL

Kendini dengeleyen ağaçlar ve kalan sorun

Kendini dengeler, derinlik $\log_2 n$

Bir milyon kayıt → yaklaşık yirmi erişim

Düğüm başına tek anahtar

Gereken: çok anahtar taşıyan düğüm

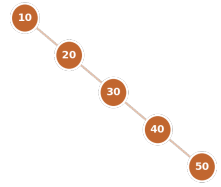
AYRIM

Bellekte ölçüt:
ziyaret edilen düğüm sayısı

Diskte ölçüt:
okunan blok sayısı

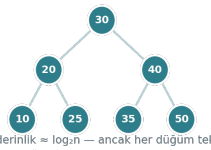
Bellek için tasarlanan çözümler diskte doğrudan kullanılamaz.

Dengesiz ikili arama ağacı



derinlik = n
en kötü durumda
sıralı taramaya döner

Dengeli (AVL) ağaç



derinlik ≈ $\log_2 n$ — ancak her düğüm tek anahtar taşır, her düğüm için ayrı bir disk erişimi gerekir.

22

AVL ağacı, ekleme ve silme işlemleri sırasında yaptığı döndürmelerle kendini dengelemekte ve ağacın derinliğini $\log_2 n$ düzeyinde tutmaktadır. Bu, dengesizleşme sorununu çözmektedir. Buna karşılık disk bağlamında yeterli bir çözüm değildir: bir milyon kayıtlık bir dosyada arama yaklaşık yirmi düğümün ziyaret edilmesini, dolayısıyla yaklaşık yirmi disk erişimini gerektirmektedir. Buradaki temel gözlem, ana bellek için tasarlanmış veri yapılarının başarım ölçütünün ziyaret edilen düğüm sayısı, disk üstü yapıların ölçütünün ise okunan blok sayısı olmasıdır. Bir düğümün tek anahtar taşıması bellekte sorun oluşturmazken, diskte her düğüm ziyareti bir blok okuması anlamına geldiğinden, blok kapasitesinin büyük bölümü kullanılmadan kalmaktadır. Bu gözlem, her düğümde birden çok anahtar taşıyan yapıların geliştirilmesine yol açmıştır.

ÇOK YOLLU AĞAÇLAR

B-tree

B-ağacı: bir düğüm = bir disk bloğu



Düğüm başına çok sayıda anahtar tutulduğundan ağacın derinliği düşer; aranan kayda birkaç blok okumasıyla erişilir.

Her düğüm bir disk bloğu

Dallanma çarpanı k → log_k(n) (k burada düğüm başına kayıt sayısı)

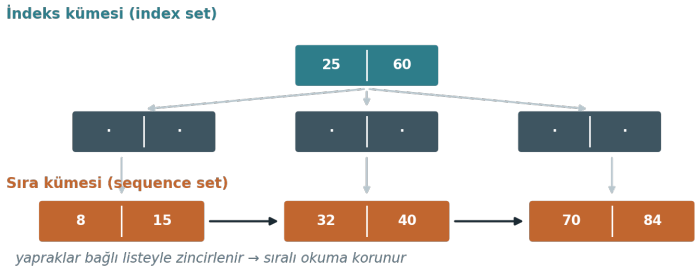
Büyüme yapraklardan köke doğru

Denge ek işlem gerektirmeden korunur

B tree, her düğümü bir disk bloğuna karşılık gelecek biçimde tasarlanmış çok yollu bir arama ağacıdır. Bir düğüm, blok kapasitesinin elverdiği ölçüde çok sayıda anahtar ve işaretçi taşımaktadır. Dallanma çarpanı k olduğunda ağacın derinliği log_k n düzeyine inmektedir. k değerinin yüz mertebesinde olduğu tipik durumlarda milyonlarca kayıtlık bir dosyada arama üç ya da dört blok okumasıyla tamamlanabilmektedir. Ağacın büyümesi yapraklardan köke doğru gerçekleştiğinden, denge ayrı bir düzenleme işlemi gerektirmeden korunmaktadır. Bir düğüm dolduğunda ikiye bölünmekte ve orta anahtar bir üst düzeye taşınmaktadır. Bu yapı, dosya yapıları alanında ana bellek çözümlerinin doğrudan aktarılması yerine ortamın kendi kısıtlarından hareket edilmesinin bir örneğidir.

SIRALI OKUMANIN KORUNMASI

B+ tree



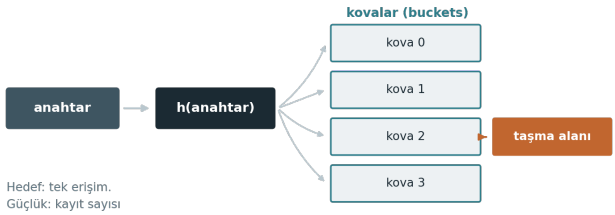
yapraklar bağlı listeyle zincirlenir → sıralı okuma korunur

- Tüm kayıtlar yaprak düzeyinde
- Yapraklar bağlı listeyle zincirli
- Hem tekil erişim hem aralık sorgusu
- İlişkisel sistemlerin varsayılan indeksi

B tree'de kayıtlar ağacın tüm düzeylerine dağılmış durumdadır. Bu durum tekil erişim için sorun oluşturmamaktadır. Ancak belirli bir aralıktaki tüm kayıtların okunması gerektiğinde ağacın dolaşılmasını zorunlu kılmaktadır. Sıralı okuma, hem raporlamada hem de aralık sorgularında sık gereken bir işlem olduğundan bu durum önemli bir eksikliktir. B+ tree, tüm kayıtları yaprak düzeyinde toplayarak ve yaprakları bağlı liste biçiminde zincirleyerek bu eksikliği gidermektedir. Üst düzeyler yalnızca yönlendirme amacıyla kullanılan ayraçları taşımaktadır. Böylece tekil erişim log_k n maliyetiyle gerçekleştirilirken, bir kayıttan sonrakine geçiş tek bir işaretçi izlemesiyle yapılabilmektedir. Bu yapı, ilişkisel veritabanı sistemlerinde de kullanılan bir indeks biçimidir.

DOĞRUDAN ADRESLEME

Tek erişim hedefi: hashing



Hedef: tek erişim.
Güçlük: kayıt sayısı değiştikçe kova düzeninin yeniden kurulması.

Anahtar → blok adresi

İdeal durumda tek disk erişimi

Sorun: çakışma (collision)

Çok hareketli sistemde olası değil

Sorun: büyüme ve yeniden yapılandırma

– Aralık sorgusu desteklenmez

Tek erişimle veriye ulaşma hedefinin doğrudan karşılığı hash tabanlı erişimdir. Bir hash fonksiyonu, anahtar değerini bir blok adresine dönüştürmektedir. Böylece herhangi bir arama yapılmadan doğrudan ilgili bloğa erişilebilmektedir. İdeal durumda maliyet tek disk erişimidir. Bu yaklaşımın iki temel güçlüğü bulunmaktadır. Birincisi çakışmadır. Farklı anahtarların aynı adrese düşmesi durumunda taşma alanları kullanılmakta ve bu da ek erişim gerektirmektedir. İkincisi, kayıt sayısının zamanla değişmesidir. Sabit boyutlu bir hash düzeninde dosya büyüdükçe taşma artmakta, bu da dosyanın yeniden yapılandırılmasını gerektirmektedir. Bu soruna karşılık geliştirilen extendible ve linear hashing yöntemleri bulunmaktadır. Ayrıca hash yapıları sıralılık bilgisini korumadığından aralık sorgularını desteklememektedir. Bu nedenle B+ ağacının yerini almaktan çok onu tamamlamaktadır.

3

Fiziksel saklama ortamları

Hiyerarşi, ödünleşimler ve kullanım alanları

SINIFLANDIRMA

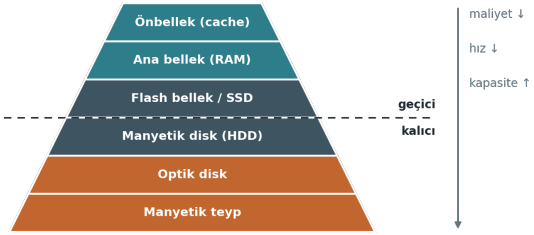
Depolama hiyerarşisi

Aşağı inildikçe: ucuz, büyük, yavaş

Ana bellek ve üzeri geçici (volatile)

Flash bellek ve altı kalıcı (non-volatile)

Bu dersin alanı: ikincil depolama



27

Saklama ortamları hız ve maliyet ölçütlerine göre bir hiyerarşi oluşturmaktadır. Hiyerarşide yukarıdan aşağıya inildikçe birim maliyet düşmekte, kapasite artmakta ve erişim süresi uzamaktadır. Bir ortam hem daha hızlı hem daha ucuz olsaydı, diğerini kullanmak için bir neden kalmazdı. Hiyerarşide ana bellek ve üzerindeki düzeyler geçici, flash bellek ve altındaki düzeyler kalıcı ortamlardır. Önbellek ile ana bellek birincil depolama, flash bellek ile manyetik disk ikincil depolama, optik disk ile manyetik teyp ise üçüncül depolama olarak adlandırılmaktadır. Bu dersin konusu ağırlıklı olarak ikincil depolama düzeyidir. Üçüncül düzey yedekleme ve arşivleme bağlamında ele alınmaktadır.

ÖZET TABLO

Ortamların karşılaştırılması

Ortam	Kalıcılık	Öne çıkan özellik	Başlıca sınırı
Önbellek	Geçici	İşlemciye en yakın düzey	Çok küçük kapasite
Ana bellek	Geçici	Doğrudan işlenebilen tek düzey	Güç kesilince kaybolur
Flash / SSD	Kalıcı	Diske göre belirgin hızlı	Sınırlı yazma-silme ömrü
Manyetik disk	Kalıcı	Yüksek kapasite, düşük maliyet	Mekanik konumlandırma
Optik disk	Kalıcı	Taşınabilir	Sınırlı kapasite
Manyetik teyp	Kalıcı	Arşivde en düşük maliyet	Yalnızca sıralı erişim

28

Tabloda, saklama ortamları kalıcılık, öne çıkan özellik ve başlıca sınır bakımından özetlenmektedir. Önbellek işlemciye en yakın düzeydir ve yönetimi donanım tarafından yapıldığından dosya yapısı tasarımının doğrudan konusu değildir. Ana bellek, üzerinde doğrudan işlem yapılabilen tek düzeydir. Ancak geçici bir ortamdır. Flash bellek diske göre belirgin biçimde hızlıdır ve kalıcıdır. Buna karşılık sınırlı bir yazma ve silme ömrüne sahiptir ve birim maliyeti diskin birkaç katıdır. Manyetik disk, yüksek kapasiteyi düşük birim maliyetle sunmaktadır. Mekanik konumlandırma nedeniyle ise yavaş kalmaktadır. Optik ortamlar taşınabilirlik sağlamakta, teyp ise arşivleme için en düşük maliyetli seçenek olmakla birlikte yalnızca sıralı erişime izin vermektedir.